DATA STRUCTURES AND ALGORITHMS

Análise de Algoritmos e Complexidade

Aula 1: Notação Big O

FIAP

Professor
Dr. Erick T. Yamamoto

Semestre
2026.2

Tópico
Eficiência e Performance

O Dilema do Programador: Além do "Funciona"

- **Falsa Sensação de Sucesso:** O código passa nos testes com 10 itens, mas trava com 1 milhão.
- **Experiência do Usuário:** Lentidão é a principal causa de abandono de sistemas e apps.
- **Custo de Infraestrutura:** Algoritmos ineficientes exigem servidores mais caros e potentes.
- **Consumo de Recursos:** Em dispositivos móveis, ineficiência significa drenagem de bateria.

"A diferença entre um algoritmo bom e um ruim não é apenas tempo..."

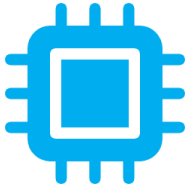
É a diferença entre ser útil ou ser inútil em escala real.

Eficiência de Algoritmos: Tempo vs. Espaço



TEMPO

Quão rápido o algoritmo executa suas instruções em relação ao tamanho da entrada.



ESPAÇO

Quanta memória RAM adicional é necessária para processar os dados.

O GRANDE EQUILÍBRIO (TRADE-OFF) DA COMPUTAÇÃO

O Problema do Hardware e do Tempo Real

Arquitetura do CPU: Diferentes processadores executam instruções em velocidades distintas.

Hierarquia de Memória: O uso de Cache L1/L2/L3 e RAM impacta drasticamente o tempo.

Linguagem e Compilador: O mesmo algoritmo em Python vs. C terá tempos reais incomparáveis.

Carga do Sistema: Processos em segundo plano podem "roubar" ciclos de CPU.

Mesmo Algoritmo, Tempos Diferentes:

Servidor High-End 64 Cores | 128GB RAM	**0.002s**
Laptop Estudante 4 Cores | 8GB RAM	**0.450s**

"Precisamos de uma métrica que ignore o hardware."

Análise Assintótica: Foco no Crescimento

Escalabilidade: Como o tempo de execução aumenta conforme os dados aumentam?

Tendência ao Infinito: Analisamos o comportamento para valores de entrada muito grandes.

Simplificação: Detalhes de hardware e constantes tornam-se irrelevantes em larga escala.

> *"Não importa se o algoritmo leva 10ms ou 20ms para 10 itens. Importa se ele leva 1 hora ou 1 segundo para 1 bilhão de itens."*

N

Tamanho da Entrada

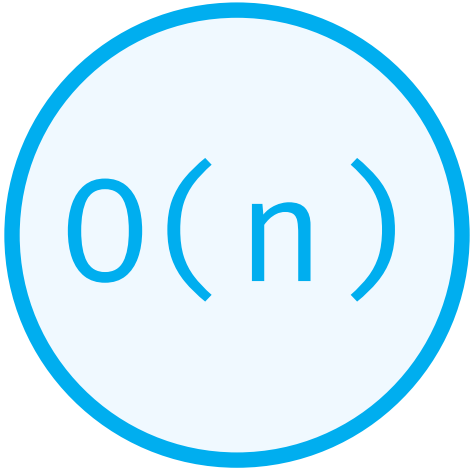
N pode representar:

- O número de elementos em um vetor.
- O número de nós em uma lista ligada.
- O número de caracteres em uma string.

Introdução à Notação Big O

Linguagem Universal
Permite que programadores comparem algoritmos independente da linguagem ou hardware.

Foco no Crescimento Não mede segundos, mas sim como o tempo de execução cresce conforme a entrada (N) aumenta.



ORDEM DE GRANDEZA

Simplificação Descarta detalhes irrelevantes e constantes para focar no comportamento assintótico.

Pior Caso Geralmente descreve o limite superior (o máximo de tempo que o algoritmo levará).

Definição Formal e Intuição Matemática

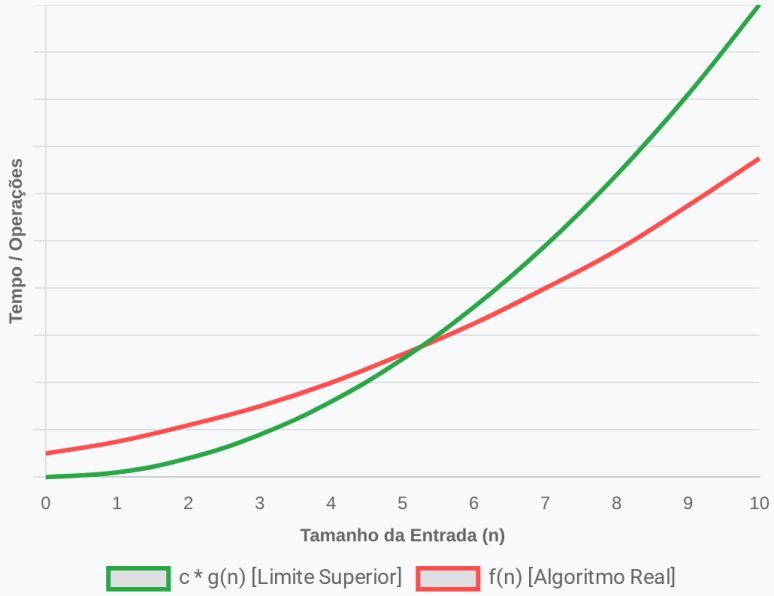
$$f(n) = O(g(n))$$

Dizemos que **f(n)** é de ordem **g(n)** se existirem constantes positivas **c** e n_0 tais que:

$$f(n) \leq c * g(n), \text{ para todo } n \geq n_0$$

Limite Superior: O Big O define um "teto". O algoritmo nunca será mais lento que esse limite para entradas grandes.

Foco no Crescimento: Não nos importamos com valores pequenos de **n**, mas sim com a tendência ao infinito.



*Visualização do Limite Superior (c * g(n))*

As Regras de Ouro da Análise Big O

1. Ignore as Constantes: Para valores de N muito grandes, multiplicadores (como 2n ou 5n) tornam-se irrelevantes.

2. Foque no Termo Dominante: Em uma expressão complexa, apenas o termo de maior grau dita o crescimento.

3. Pense no Infinito: A análise assintótica foca no comportamento do algoritmo quando N tende ao infinito.

Exemplos de Simplificação:

$O(\cancel{2}n + \cancel{10})$	→	$O(n)$
$O(n^2 + \cancel{5n + 1})$	→	$O(n^2)$
$O(\cancel{100}n \log n + \cancel{n})$	→	$O(n \log n)$

O(1) - Complexidade Constante

Tempo Fixo: O tempo de execução não muda, independentemente do tamanho da entrada (N).

Performance Ideal: É a classe de complexidade mais eficiente possível.

Exemplos Comuns:
- Acessar um elemento de um vetor pelo índice.
- Operações aritméticas (+, -, *, /).
- Atribuição de variáveis.
- Comparação de dois números.

ACESSO A ARRAY

```
int valor = vetor[500]; // O(1)
```

OPERAÇÃO MATEMÁTICA

```
int resultado = (a + b) * 10; // O(1)
```

FUNÇÃO SIMPLES

```
void imprimirPrimeiro(int arr[]) {
printf("%d", arr[0]); // O(1)
}
```

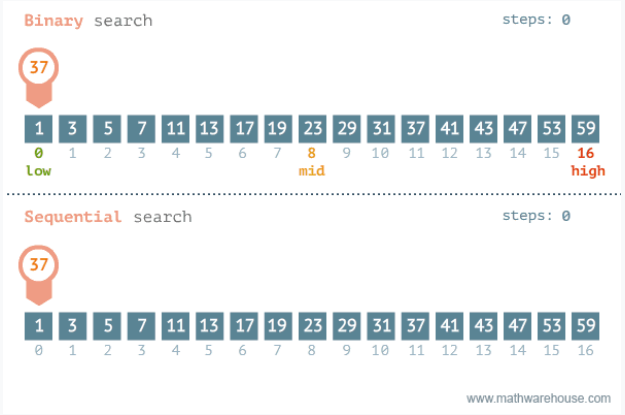
O(log n) - Complexidade Logarítmica

- **Divisão e Conquista:** O tamanho do problema é reduzido pela metade a cada iteração.
- **Alta Eficiência:** Extremamente rápido mesmo para entradas massivas.
- **Exemplo Clássico:** Busca Binária em um vetor ordenado.
- **Crescimento Lento:** O tempo aumenta muito pouco conforme N cresce exponencialmente.

Visualizando a Busca Binária:

Para **N = 1.000.000** itens,

são necessários apenas **~20 passos**!



O(n) - Complexidade Linear

```c
// Percorrendo um vetor de tamanho N
void exibirVetor(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
}
```

O tempo de execução cresce na mesma proporção que N.

- **Proporcionalidade:** Se dobrarmos o tamanho da entrada (N), o tempo de execução também dobra.
- **Laços Simples:** Comum em algoritmos que precisam visitar cada elemento da entrada uma única vez.
- **Exemplos:** Busca linear, soma de elementos, encontrar o maior valor em um vetor.

CRESCIMENTO LINEAR

O(n log n) - Complexidade Linear-Logarítmica

- **Padrão Ouro:** É a complexidade mais eficiente possível para algoritmos de ordenação baseados em comparação.
- **Divisão e Conquista:** Ocorre quando dividimos o problema em partes menores (log n) e processamos cada parte linearmente (n).
- **Performance em Escala:** Muito superior ao O(n²) para grandes volumes de dados.

Merge Sort **Quick Sort** **Heap Sort**

A Matemática da Eficiência:

$$O(n * \log n)$$

Imagine um loop que percorre **N** elementos, mas dentro dele, a cada iteração, o espaço de busca é reduzido pela metade (**log N**).

Essencial para Big Data e sistemas de alta performance.

O(n²) - Complexidade Quadrática

```c
// Exemplo: Comparando todos os elementos (N x N)
void compararTodos(int arr[], int n) {
    for (int i = 0; i < n; i++) { // Loop Externo
        for (int j = 0; j < n; j++) { // Loop Interno
          // Operação O(1) executada N * N vezes
            printf("%d-%d ", arr[i], arr[j]);
        }
    }
}
```

PERIGO: Loops Aninhados!

O tempo de execução cresce com o **quadrado** da entrada. Dobrar N significa quadruplicar o tempo.

Entrada (N)	Operações (N²)
10	100
100	10.000
1.000	1.000.000
100.000	10.000.000.000

Comum em algoritmos de ordenação simples como Bubble Sort e Selection Sort.

$O(2^n)$ e $O(n!)$ - Explosão Combinatória

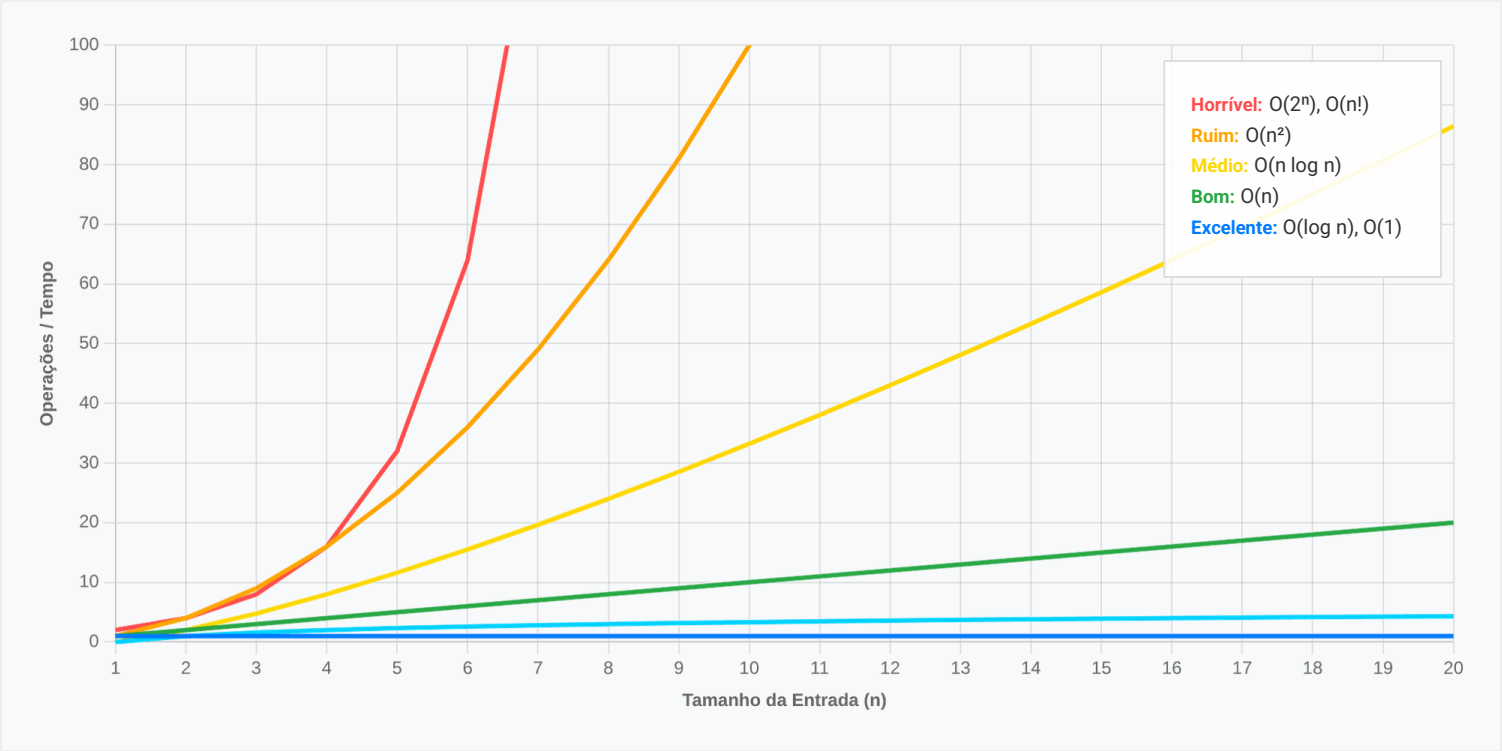
$O(2^n)$ - Exponencial: Comum em recursão sem memorização (ex: Fibonacci ingênuo). O tempo dobra a cada novo elemento.

$O(n!)$ - Fatorial: O pior cenário possível. Comum em problemas de permutação (ex: Caixeiro Viajante por força bruta).

"Algoritmos que funcionam para N=10, mas levariam a idade do universo para N=100."

N	$O(n)$	$O(n^2)$	$O(2^n)$
10	10 ops	100 ops	1.024 ops
20	20 ops	400 ops	1.048.576 ops
50	50 ops	2.500 ops	1.125.899.906.842.624 ops
100	**100 ops**	**10.000 ops**	**Inviável (Idade do Universo)**

Gráfico Comparativo de Complexidade



Análise Prática 1: Loop Simples

```
int somarVetor(int arr[], int n) {
   int soma =  0;   O(1)
   for( int i =  0; i < n; i++) {    O(n)
      soma += arr[i];    O(1)
    }
   return soma;    O(1)
}
```

Passo 1: Operações Atômicas

Atribuições e retornos fora de laços são constantes: **O(1)**.

Passo 2: O Laço (Loop)

O loop executa **N** vezes. A operação interna é O(1), logo o bloco do loop é **N * O(1) = O(n)**.

Passo 3: Somatório

Total = O(1) + O(n) + O(1). Aplicando a regra do termo dominante...

COMPLEXIDADE FINAL:

O(n)

Análise Prática 2: Loops Aninhados

Loops Independentes

```
for (int i=0; i<n; i++) {
// O(n)
}
 for (int j=0; j<m; j++) {
// O(m)
}
```

Regra da Soma:
O(n + m) → **O(max(n, m))**

Quando os loops estão um após o outro, somamos suas complexidades.

Loops Aninhados

```
for (int i=0; i<n; i++) {
for (int j=0; j<n; j++) {
// O(n * n)
}
}
```

Regra do Produto:
O(n * n) → **O(n²)**

Quando um loop está dentro de outro, multiplicamos suas complexidades.

Análise Prática 3: Condicionais e Funções

```
void processarDados(int n, bool rapido) {
    if (rapido) {
       printf("OK");    O(1)
    } else {
       for( int i=0; i<n; i++) {      O(n)
            printf("%d", i);
        }
    }
}

// Qual a complexidade desta função?
```

Regra do Pior Caso

Em estruturas condicionais (if/else), a complexidade total é definida pelo **caminho mais caro**.

Chamadas de Função

Se uma função chama outra, somamos ou multiplicamos a complexidade da função chamada ao fluxo principal.

Complexidade Final: O(n)
(Pois O(n) domina O(1))

Melhor, Médio e Pior Caso (Worst Case)



Melhor Caso

O cenário ideal. Ex: Encontrar o item buscado na primeira posição do vetor.
Notação: Ω (Omega)



Caso Médio

O desempenho esperado na maioria das vezes. Ex: Encontrar o item no meio do vetor.
Notação: Θ (Theta)



Pior Caso

O limite superior de tempo. Ex: O item não existe ou está na última posição.
Notação: O (Big O)

POR QUE FOCO NO PIOR CASO? Porque ele oferece uma GARANTIA de performance.
Sabemos que o sistema nunca será mais lento que o definido pelo Big O.

Complexidade de Espaço (Memória)

Definição: Mede a quantidade de memória RAM adicional que um algoritmo necessita para ser executado.

Espaço Auxiliar: Focamos na memória extra usada pelo algoritmo, ignorando o espaço ocupado pela entrada original.

Recursão: Chamadas recursivas consomem espaço na *Stack* (Pilha), o que pode levar a O(n) de espaço.

Exemplos de Alocação:

```
// Espaço Constante O(1)
int temp = 0;
for(int i=0; i<n; i++) { ... }
```

```
// Espaço Linear O(n)
int* copia = malloc(n * sizeof(int));
```

Assim como no tempo, usamos Big O para descrever o crescimento do uso de memória.

Trade-off: Tempo vs. Espaço

Priorizando Tempo

+ **Estratégia:** Usar memória extra para evitar reprocessamento.

✓ **Exemplo:** Memoização (armazenar resultados de funções caras).

✓ **Exemplo:** Tabelas Hash (acesso O(1) gastando mais RAM).

→ **Resultado:** Algoritmo mais rápido, maior consumo de memória.

Priorizando Espaço

+ **Estratégia:** Processar dados no local (in-place) sem cópias extras.

✓ **Exemplo:** Algoritmos iterativos simples em vez de recursivos.

✓ **Exemplo:** Compactação de dados durante o processamento.

→ **Resultado:** Menor uso de RAM, possivelmente mais lento.

A escolha depende do ambiente: Sistemas Embarcados (Espaço) vs. Web/Cloud (Tempo).

Atividade em Sala: Análise de Código

Determine a complexidade Big O para cada um dos trechos abaixo:

Trecho 1

```
for (int i=0; i<n; i++) {
printf("%d", i);
}
for (int j=0; j<n; j++) {
printf("%d", j);
}
```

O(?)

Trecho 2

```
for (int i=0; i<n; i++) {
for (int j=0; j<n; j++) {
soma += i * j;
}
}
```

O(?)

Trecho 3

```
int i = n;
while (i > 1) {
i = i / 2;
printf("%d", i);
}
```

O(?)

Desafio para Casa: Busca Linear vs. Binária

O Desafio da Performance

Implemente um programa em C que comprove na prática a diferença de eficiência entre as complexidades `O(n)` e `O(log n)`.

- Crie um vetor ordenado com **1.000.000** de elementos inteiros.
- Implemente as funções `buscaLinear()` e `buscaBinaria()`.
- Utilize a biblioteca `<time.h>` para medir o tempo de execução de cada busca.
- Realize buscas pelo **último elemento** do vetor (Pior Caso).
- Exiba no console o tempo gasto por cada algoritmo em milissegundos.
- Escreva um breve comentário no código explicando os resultados obtidos.

Resumo e Próxima Aula

O que aprendemos hoje:

- A importância da eficiência (Tempo vs Espaço).
- Notação Big O como métrica universal.
- Classes de complexidade: O(1) até O(n!).
- Análise prática de loops e condicionais.
- O conceito de Pior Caso (Worst Case).



Aula 2

Recursão e Estruturas Lineares

Vamos aprender a resolver problemas complexos dividindo-os em subproblemas menores e como analisar a complexidade de funções que chamam a si mesmas.